Отчёт о выполнении задания по параллельному программированию

Вариант 703

Глущенко Анна Сергеевна, 327 группа

В данном практическом задании были реализованы две версии параллельной программы с использованием технологии OpenMP и параллельная программа с использованием технологии MPI. В качестве исходного кода использовалась программа, реализующая итерационный метод релаксации.

Исходная программа

Размер задачи N задается через макрос. Для удобства сравнения времени работы программ на разных размерах массива, он был вынесен в отдельный заголовочный файл `params.h`.

Время выполнения программы засекалось с помощью функции `omp_get_wtime()`.

Код исходной программы приведен в файле `original.c`.

```c
#include <omp.h>
#include "params.h"

#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#define  Max(a,b)  ((a)>(b)?(a):(b))

double maxeps = 0.1e-7;
int itmax = 100;
int i,j,k;
double eps;
double A [N][N][N], B [N][N][N];

void relax();
void resid();
void init();
void verify();

int main(int an, char **as)
{
    printf("===== Original program =====\n\n");
    double start = omp_get_wtime();

    int it;
    init();
    for(it=1; it<=itmax; it++)
    {
        eps = 0.;
```

```
        relax();
        resid();
        printf( "it=%4i   eps=%f\n", it,eps);
        if (eps < maxeps) break;
    }
    verify();

    double end = omp_get_wtime();
    double result_time = (end - start) * 1000;
    printf("\nExecution time: %lf ms\n", result_time);

    return 0;
}

void init()
{
    for(k=0; k<=N-1; k++)
    for(j=0; j<=N-1; j++)
    for(i=0; i<=N-1; i++)
    {
        if(i==0 || i==N-1 || j==0 || j==N-1 || k==0 || k==N-1)
A[i][j][k]= 0.;
        else A[i][j][k]= (4. + i + j + k);
    }
}

void relax()
{
    for(k=2; k<=N-3; k++)
    for(j=2; j<=N-3; j++)
    for(i=2; i<=N-3; i++)
    {
        B[i][j][k]=(A[i-1][j][k]+A[i+1][j][k]+A[i][j-
1][k]+A[i][j+1][k]+A[i][j][k-1]+A[i][j][k+1]+
            A[i-2][j][k]+A[i+2][j][k]+A[i][j-
2][k]+A[i][j+2][k]+A[i][j][k-2]+A[i][j][k+2])/12.;
    }
}

void resid()
{
    for(k=1; k<=N-2; k++)
    for(j=1; j<=N-2; j++)
    for(i=1; i<=N-2; i++)
    {
        double e;
        e = fabs(A[i][j][k] - B[i][j][k]);
        A[i][j][k] = B[i][j][k];
        eps = Max(eps,e);
    }
}
```

```
void verify()
{
    double s;
    s=0.;
    for(k=0; k<=N-1; k++)
    for(j=0; j<=N-1; j++)
    for(i=0; i<=N-1; i++)
    {
        s=s+A[i][j][k]*(i+1)*(j+1)*(k+1)/(N*N*N);
    }
    printf("  S = %f\n",s);
}
```

Оптимизация последовательной программы

Для оптимизации работы изначальной программы, был поменян порядок обхода циклов с `k, j, i` на `i, j, k`, что обеспечивает локальность доступа к данным, переменные-счётчики циклов объявлены локально, это позволит компилятору оптимизировать доступ к ним, так как они обычно располагаются на регистрах процессора. Оптимизированная программа представлена в файле `original_optimized.c`.

В таблице 1 приведено сравнение времени работы (в секундах) исходной программы и программы после оптимизации (тестирования проводились на суперкомпьютере Polus).

Таблица 1. Сравнение оптимизированной и неоптимизированной последовательных программ

N	Время до оптимизации	Время после оптимизации
66	1.926	1.208
130	35.189	9.973
258	554.095	92.742

Далее сравнения параллельной программы проводились с оптимизированной последовательной.

Технология OpenMP

Распараллеливание программы с помощью директивы for

В файле `parallel.c` приведен код параллельной версии программы с использованием директивы `for`.

Опытным путем было решено использовать директиву `collapse(2)` для распределения работ внешних циклов и `simd` для внутреннего. Это даёт некоторый выигрыш по времени (тестирования проводились локально на массиве 258*258*258).

Параллельная область внутри цикла итераций создается для двух функций сразу (для `relax` и `resid`), чтобы минимизировать накладные расходы. Также было выявлено, что динамическое распределение витков цикла даёт самое оптимальное время.

Итоговая прагма для функций без редукции по переменным:
`#pragma omp parallel for collapse(2) schedule(dynamic)`

В функциях `resid` и `verify` использовалась редукция по переменным `eps` и `s` соответственно: `reduction(max:eps)` и `reduction(+:s)`.

Замеры времени работы программы на разном количестве нитей и объёме входных данных представлены в таблице 2.

Таблица 2. Время работы параллельной программы (for)

Нити \ N	**66**	**130**	**258**	**514**
1	1.228	10.225	92.608	901.730
2	0.652	5.344	47.559	751.730
3	0.448	3.421	32.424	254.868
4	0.333	2.802	24.051	192.886
5	0.275	2.220	19.915	154.284
6	0.232	1.856	16.491	128.874
7	0.203	1.636	15.269	110.700
8	0.183	1.478	14.639	96.828
9	0.168	1.459	12.861	86.761
10	0.169	1.378	11.332	85.412
20	0.136	0.930	7.663	44.894
40	0.127	0.860	6.832	36.944
60	0.128	0.749	5.579	33.266
80	0.161	0.724	4.395	30.189
100	0.208	0.620	3.942	29.267
120	0.240	0.578	3.900	28.379
140	0.285	0.500	3.880	28.038
160	0.223	0.450	3.750	28.021

Зависимость ускорения от числа потоков ($S = \frac{T_{\text{последовательной}}}{T_{\text{параллельной}}}$) на различных входных данных показана на рис. 1.

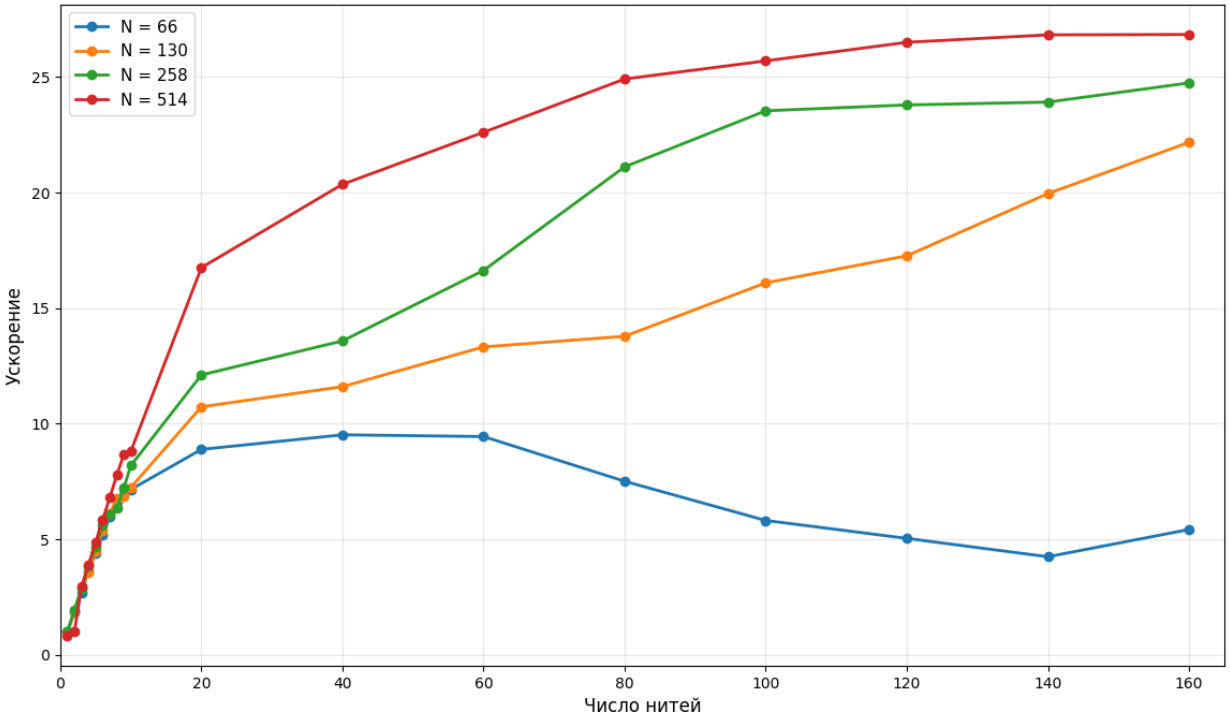


Рис. 1. 2-d график зависимости ускорения от числа нитей

Видно, что ускорения для N = 66 «проседает» на 60 нитях. Предположительно это связано большими накладными расходами на порождение нитей при относительно небольших входных данных.

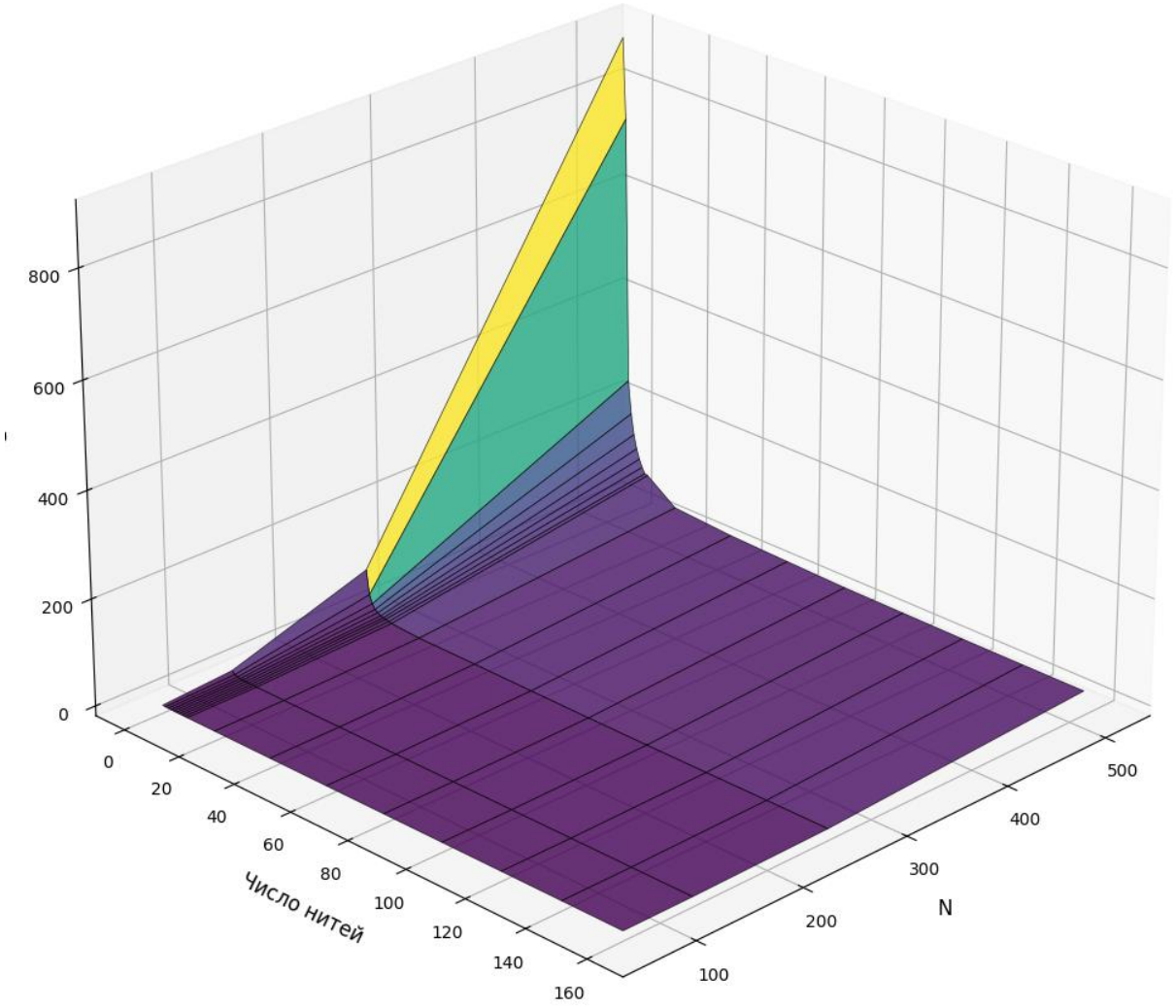


Рис. 2. 3-d график зависимости времени выполнения от числа нитей и размера входных данных

Распараллеливание программы с помощью механизма задач

В файле `parallel_task.c` приведен код параллельной версии программы с использованием механизма задач.

В этой реализации можно вынести создание параллельной области за пределы итерационного цикла в функции `main` и порождать задачи одной нитью.

```
#pragma omp parallel
    {
        #pragma omp master
        {
            init();
            for(int it=1; it<=itmax; it++)
            {
                eps = 0.;
                relax();
                resid();
                printf( "it=%4i   eps=%f\n", it, eps);
                if (eps < maxeps) break;
            }
            verify();
```

```
        }
    }
```

Было выяснено, что программа с директивами `taskloop` работает очень медленно (по скорости сравнимо с последовательной программой). Код этой версии приведен ниже.

```
#include <omp.h>
#include "params.h"

#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#define  Max(a,b) ((a)>(b)?(a):(b))

double maxeps = 0.1e-7;
int itmax = 100;
double eps;
double s = 0.;
double A [N][N][N] __attribute__((aligned(64)));
double B [N][N][N] __attribute__((aligned(64)));

void relax();
void resid();
void init();
void verify();

int main(int an, char **as)
{
    printf("===== Parallel program on tasks =====\n\n");
    double start = omp_get_wtime();

    int flag = 0;

    #pragma omp parallel
    {
        #pragma omp master
        {
            init();
            #pragma omp taskwait

            for(int it=1; it<=itmax; it++)
            {
                eps = 0.;

                relax();
                #pragma omp taskwait

                resid();
                #pragma omp taskwait

                printf( "it=%4i   eps=%f\n", it, eps);
```

```
                if (eps < maxeps) break;
            }

            verify();
        }
    }

    double end = omp_get_wtime();
    double result_time = (end - start) * 1000;
    printf("\nExecution time: %lf ms\n", result_time);

    return 0;
}

void init()
{
    #pragma omp taskloop collapse(2)
    for(int i=0; i<=N-1; i++)
    for(int j=0; j<=N-1; j++)
    #pragma omp simd aligned(A,B:64)
    for(int k=0; k<=N-1; k++)
    {
        if(i==0 || i==N-1 || j==0 || j==N-1 || k==0 || k==N-1) A[i][j][k]= 0.;
        else A[i][j][k]= (4. + i + j + k);
    }
}

void relax()
{
    #pragma omp taskloop collapse(2)
    for(int i=2; i<=N-3; i++)
    for(int j=2; j<=N-3; j++)
    #pragma omp simd aligned(A,B:64)
    for(int k=2; k<=N-3; k++)
    {
        B[i][j][k]=(A[i-1][j][k]+A[i+1][j][k]+A[i][j-1][k]+A[i][j+1][k]+A[i][j][k-
1]+A[i][j][k+1]+
            A[i-2][j][k]+A[i+2][j][k]+A[i][j-2][k]+A[i][j+2][k]+A[i][j][k-
2]+A[i][j][k+2])/12.;
    }
}

void resid()
{
    #pragma omp taskloop collapse(2) reduction(max:eps)
    for(int i=1; i<=N-2; i++)
    for(int j=1; j<=N-2; j++)
    #pragma omp simd aligned(A,B:64)
    for(int k=1; k<=N-2; k++)
    {
        double e;
```

```
        e = fabs(A[i][j][k] - B[i][j][k]);
        A[i][j][k] = B[i][j][k];
        eps = Max(eps,e);
    }
}

void verify()
{
    s = 0.;

    #pragma omp taskloop collapse(2) reduction(+:s)
    for(int i=0; i<=N-1; i++)
    for(int j=0; j<=N-1; j++)
    #pragma omp simd aligned(A:64)
    for(int k=0; k<=N-1; k++)
    {
        s=s+A[i][j][k]*(i+1)*(j+1)*(k+1)/(N*N*N);
    }

    printf("  S = %f\n",s);
}
```

Для оптимизации работы с механизмом задач, было решено на каждую задачу выделять некоторый блок цикла. Размер блока `BLOCK_SIZE` меняется в зависимости от размера входных данных. Объявление этого макроса представлено в заголовочном файле `params.h` и выглядит следующим образом.

```
#if N <= 66
    #define BLOCK_SIZE 8
#elif N <= 130
    #define BLOCK_SIZE 16
#elif N <= 258
    #define BLOCK_SIZE 32
#else
    #define BLOCK_SIZE 64
#endif
```

Также для функций `resid` и `verify` реализована редукция глобальных переменных с помощью редукционного массива. Пример для `resid` приведен ниже.

```
void resid()
{
    int t_count = omp_get_max_threads();
    double eps_loc[t_count];
    for (int i = 0; i < t_count; i++) eps_loc[i] = 0.0;

    for(int i=1; i<=N-2; i+=BLOCK_SIZE)
    for(int j=1; j<=N-2; j+=BLOCK_SIZE)
    for(int k=1; k<=N-2; k+=BLOCK_SIZE)
    {
```

```
        #pragma omp task firstprivate(i,j,k) shared(eps_loc)
        {
            int tid = omp_get_thread_num();
            double e_loc = 0.0;

            for (int ii = i; ii < Min(i+BLOCK_SIZE, N-1); ii++)
            for (int jj = j; jj < Min(j+BLOCK_SIZE, N-1); jj++)
            for (int kk = k; kk < Min(k+BLOCK_SIZE, N-1); kk++)
            {
                double e = fabs(A[ii][jj][kk] - B[ii][jj][kk]);
                A[ii][jj][kk] = B[ii][jj][kk];
                e_loc = Max(e_loc, e);
            }

            eps_loc[tid] = Max(e_loc, eps_loc[tid]);
        }
    }
    #pragma omp taskwait

    for (int i = 0; i < t_count; i++)
        if (eps_loc[i] > eps) eps = eps_loc[i];
}
```

Замеры времени работы программы на разном количестве нитей и объеме входных данных представлены в таблице 3.

Таблица 3. Время работы параллельной программы (task)

Нити \ N	66	130	258	514
1	1.356	10.687	97.068	803.621
2	0.720	5.601	50.133	405.890
3	0.503	3.765	34.192	272.512
4	0.395	2.899	25.691	205.740
5	0.314	2.371	21.075	167.733
6	0.265	2.029	17.499	138.190
7	0.232	1.745	15.209	120.876
8	0.209	1.572	13.544	117.855
9	0.196	1.483	12.661	111.359
10	0.207	1.269	10.882	100.854
20	0.289	0.960	6.803	72.720
40	0.306	0.790	5.675	53.603
60	0.283	0.786	5.696	50.509
80	0.308	0.815	5.224	56.331
100	0.333	0.794	4.323	31.722
120	0.397	0.806	4.030	30.243
140	0.427	0.824	3.910	29.544
160	0.322	0.537	3.834	29.498

На рис. 3 и 4 показаны график зависимость ускорения от числа потоков для различных входных данных и трехмёрных график зависимости времени выполнения от количества нитей и размера входных данных.

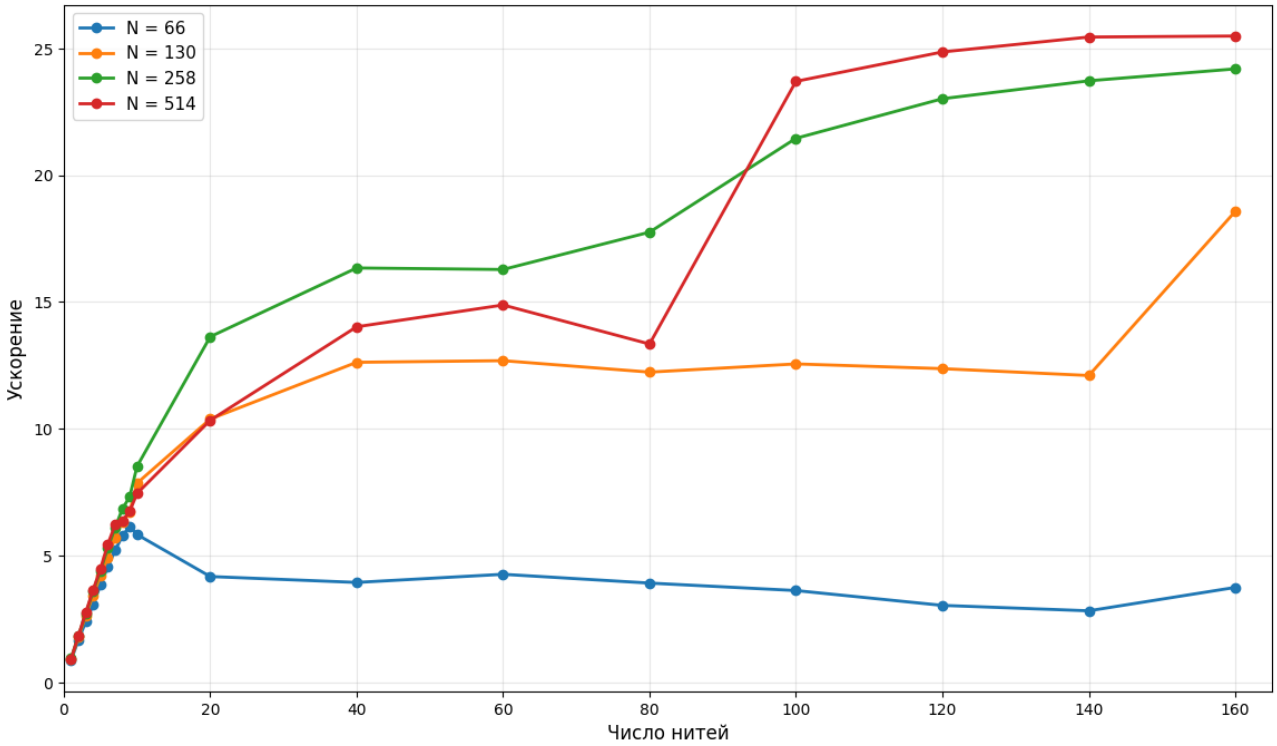


Рис. 3. 2-d график зависимости ускорения от числа нитей

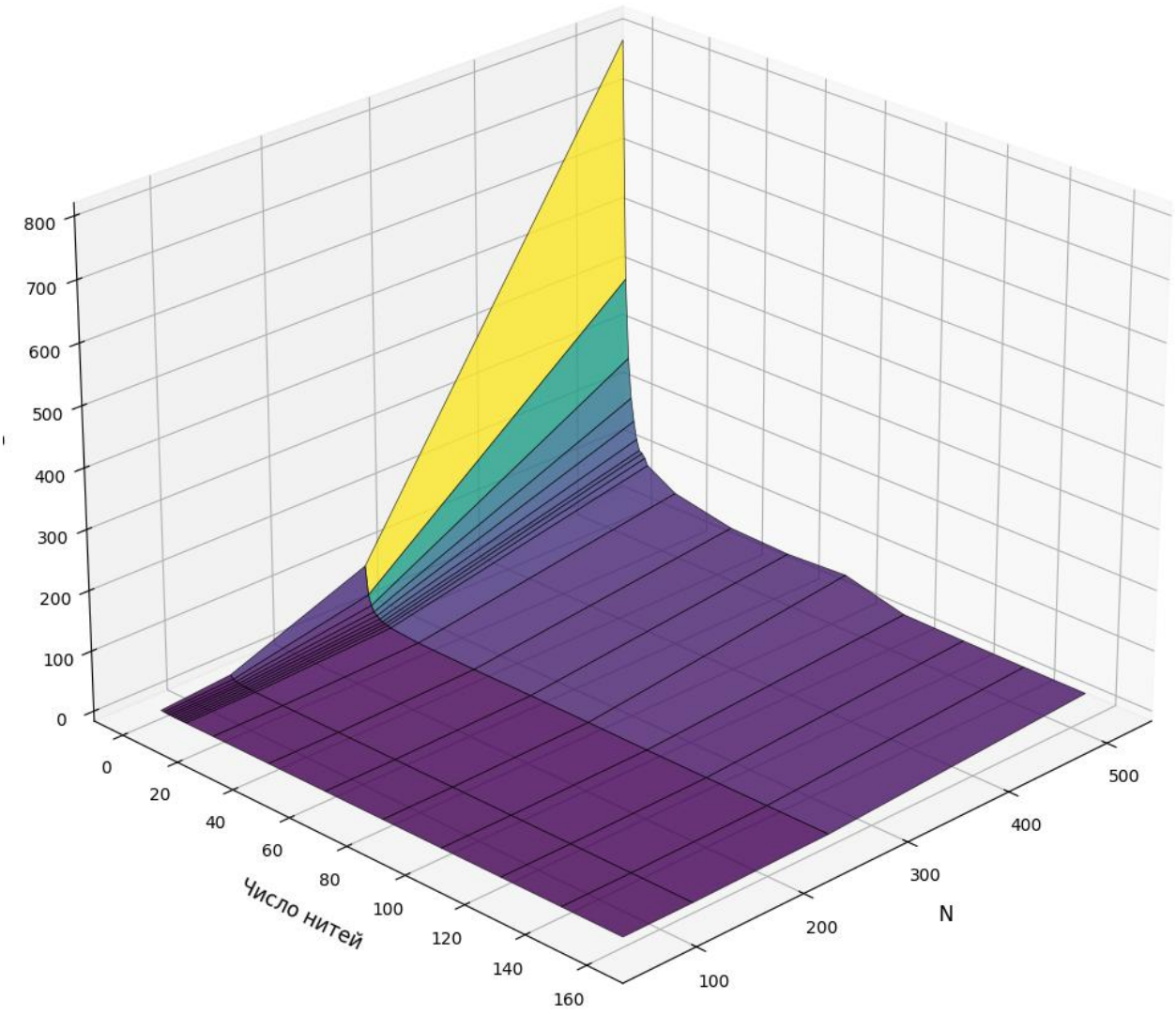


Рис. 4. 3-d график зависимости времени выполнения от числа нитей и размера входных данных

Технология MPI

В файле `mpi.c` приведен код параллельной версии программы. В ней, как и с предыдущих программах определение константы `N` вынесено в отдельный заголовочный файл `params.h`.

В функции main происходит инициализация MPI.

```
MPI_Init(&an, &as);
MPI_Comm_size(MPI_COMM_WORLD, &proc);
MPI_Comm_rank(MPI_COMM_WORLD, &proc_num);
MPI_Barrier(MPI_COMM_WORLD);
```

Так как используется технология MPI, то необходимо было распределить данные между процессами. В данной программе обрабатывается трехмерный массив, поэтому было решено делить его между процессами по слоям по первому измерению. Массивы выделяются динамически и инициализируются в функции `init()`, кроме того для каждого массива выделяются теневые грани (по 2 слоя с каждой стороны), в которых хранятся данные для обмена между процессами.

```
startslice = round(((double)N / proc) * proc_num);
endslice = round(((double)N / proc) * (proc_num + 1));
```

```
nslice = endslice - startslice;

A = calloc(N * N * (nslice + 4), sizeof(double));
B = calloc(N * N * (nslice + 4), sizeof(double));
```

Переменные `proc, proc_num` хранят количество процессов и номер конкретного процесса, `startslice, endslice, nslice` отвечают за границы распределенного в данный процесс массива.

Поскольку распределенные массивы имеют теневые грани, для удобства работы с ними были определены следующие макросы.

```
#define A(i,j,k) A[((i)+2)*N*N + (j)*N + (k)]
#define B(i,j,k) B[((i)+2)*N*N + (j)*N + (k)]
```

Собственно решение задачи происходит в функции `relax()`. Перед вызовом основного цикла функции происходит обмен теневыми гранями и ожидание завершения обмена. Были использованы неблокирующие обмены `MPI_Irecv, MPI_Isend` во избежание deadlock и для большей эффективности.

```
if (proc_num != 0) {
    MPI_Irecv(&A(-2,0,0), N*N*2, MPI_DOUBLE, proc_num-1, 0, MPI_COMM_WORLD,
&recv[0]);
}
if (proc_num != proc - 1) {
    MPI_Isend(&A(nslice-2,0,0), N*N*2, MPI_DOUBLE, proc_num+1, 0, MPI_COMM_WORLD,
&send[1]);
}
if (proc_num != proc - 1) {
    MPI_Irecv(&A(nslice,0,0), N*N*2, MPI_DOUBLE, proc_num+1, 1, MPI_COMM_WORLD,
&recv[1]);
}
if (proc_num != 0) {
    MPI_Isend(&A(0,0,0), N*N*2, MPI_DOUBLE, proc_num-1, 1, MPI_COMM_WORLD,
&send[0]);
}

if (proc_num == 0) {
    MPI_Wait(&recv[1], &stat[0]);
} else if (proc_num == proc - 1) {
    MPI_Wait(&recv[0], &stat[1]);
} else {
    MPI_Waitall(2, recv, stat);
}
```

Остальные функции практически не изменялись, за исключением того, что была реализована редукция глобальной переменной `eps` и общей переменной `s`.

```
MPI_Allreduce(&local_eps, &eps, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
```

```
double global_s = 0.;
MPI_Allreduce(&s, &global_s, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
```

Замеры времени работы программы на разном количестве процессов и объеме входных данных представлены в таблице 4.

Таблица 4. Время работы параллельной программы (MPI)

Процессы \ N	66	130	258	514
1	0.849739	7.241950	56.520395	893.645480
2	0.426632	5.970107	28.347253	377.027824
3	0.312311	4.183344	19.977837	205.325334
4	0.299901	3.145362	22.845750	242.969169
5	0.235059	2.171650	12.402312	240.657282
6	0.237939	1.520896	10.426447	136.493513
7	0.225275	1.296456	9.158346	111.597472
8	0.257701	1.092303	8.136701	116.535929
9	0.233458	0.966409	7.684370	83.114609
10	0.212407	0.902480	7.108533	70.678671
20	0.202335	0.664989	5.718706	53.655796
40	0.124882	0.687979	5.795146	33.239395
60	0.103912	0.703143	4.385710	30.804617
80	0.093088	0.559353	7.389643	27.299053
100	0.096599	0.589141	8.527663	30.950796
120	0.114308	0.661266	8.607724	51.576651
140	0.125281	0.477088	3.908471	35.976760

На рис. 5 и 6 показаны график зависимость ускорения от числа процессов для различных входных данных и трехмёрных график зависимости времени выполнения от количества процессов и размера входных данных.

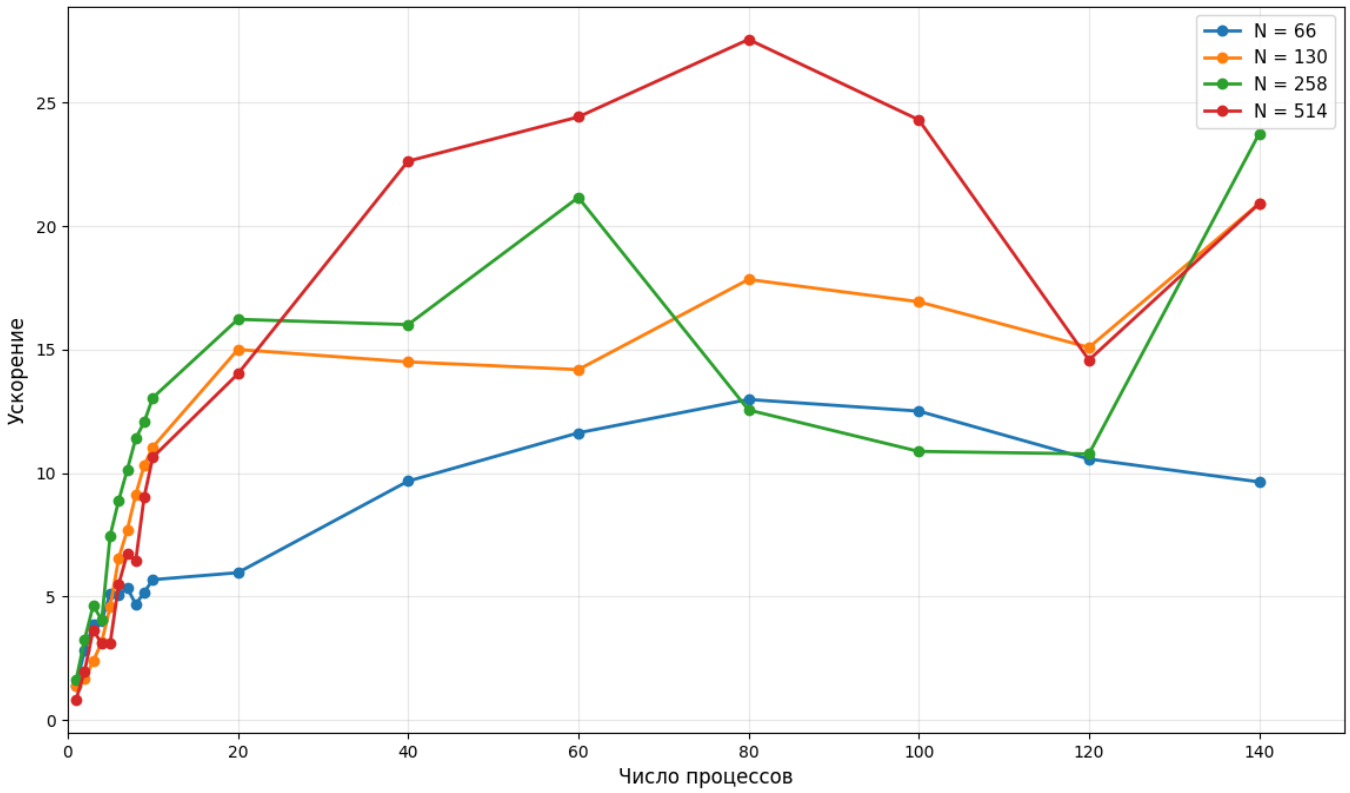


Рис. 5. 2-d график зависимости ускорения от числа процессов

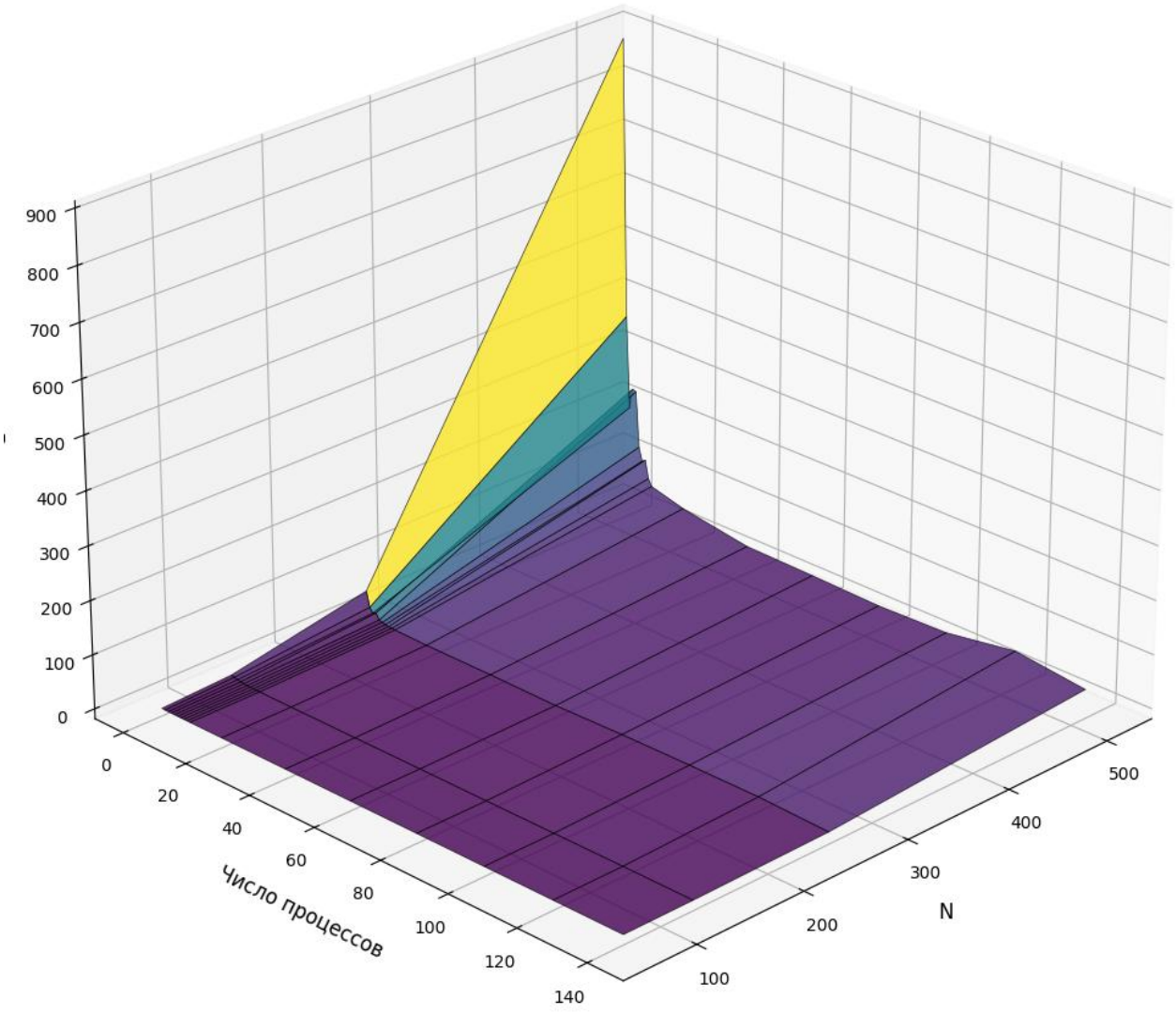


Рис. 6. 3-d график зависимости времени выполнения от числа процессов и размера входных данных

Выводы

При анализе графиков двух версий параллельной программы, реализованных с помощью OpenMP, можно прийти к следующим заключениям.

1. Директива `for` по сравнению с механизмом задач проще в реализации и даёт равномерное ускорение на всех размерах данных.
2. Оптимальное количество потоков в зависимости от входных данных:
   - $N = 66$ – до 20 потоков
   - $N = 130$ – 40-60 потоков
   - $N = 258$ – 100 потоков
   - $N = 514$ – 140-160 потоков
3. Для малых задач наблюдается «проседание» производительности при большом числе потоков. Это может быть связано с накладными расходами на порождение нитей, неравномерную загрузку процессора.

Анализируя график ускорений для параллельной программы, реализованной с помощью технологии MPI, можно сказать следующее.

1. При малом числе процессов наблюдается сильный рост ускорения программы.
2. Для малых объемов задачи ускорение быстро выходит на плато, и дальнейшее увеличение числа процессов только ухудшает время выполнения. Это говорит о том, что эффект, который дает параллельная программа, не покрывает накладные расходы.

Причинами замедления выполнения программы на большом числе процессов может быть дисбаланс нагрузки, недетерминированности работы компьютера, накладных расходов на обмен данными, синхронизацию. Поэтому каждая задача требует подбора оптимальных параметром машины.

В сравнении с MPI OpenMP существенно проще в реализации, не требует распределения данных. По графикам реализации получились похожими по эффективности, обе они показывают замедление при большом числе процессов. Наибольший потенциал имеет комбинированный подход MPI+OpenMP к распараллеливанию, например, распределение слоев по процессам и параллельная обработка каждого слоя внутри процесса.